MCO-1D ORNITHOPTER

Complete Commissioning and Operating Manual

Bench Tests · TC-02 · TC-03 · Ground Station · First Flight · Revision 3

For personnel who are new to this project. Every step explained from first principles. Nothing assumed.

CONFIDENTIAL — Programme Document

TABLE OF CONTENTS

1. What This Aircraft Is and What It Does
2. Hardware Inventory — Every Part Listed
3. Electronics Wiring — What Connects to What
4. Software — What It Is and How It Works
5. Setting Up the Raspberry Pi
6. Loading the Wave Library onto the Pi
7. Setting Up the Laptop Ground Station
8. ODrive Configuration — Step by Step
9. Pixhawk Configuration — Step by Step
10. Servo Configuration and Calibration
11. TC-02 Static Thrust Test — Complete Procedure
12. TC-03 Dynamic Thrust Test — Complete Procedure
13. First Free Flight — Checklist and Procedure
14. What the Optimizer Is Doing and How to Read It
15. Emergency Procedures
16. Glossary — Every Technical Term Explained
17. Gust Response System — GR-1 through GR-8
18. New Flight Modes — Backward, Inclined Hover, Inverted Recovery
19. Software Updates and Performance
20. High-Speed Sprint and Progressive Wing Stowing
21. Autonomous Mode Discovery and Lever Evolution
22. Loop 9 — Meta-Optimizer

SECTION 1 — WHAT THIS AIRCRAFT IS AND WHAT IT DOES

1.1 The Basic Idea

The MCO-1D is an ornithopter — an aircraft that flies by flapping its wings, like a bird or a dragonfly, rather than using a propeller or jet engine. It has four pairs of wings (eight wings in total) that beat up and down continuously while in flight. The pattern in which the wings beat — how fast, how far, and with what timing between pairs — is controlled by a computer running software called the Wave Library.

Unlike a conventional aircraft where the engine produces thrust and the wings produce lift as separate functions, on the MCO-1D the flapping wings do both simultaneously. Flapping faster produces more lift and more thrust. Changing how far the wings travel on each stroke changes the aerodynamic character of the flight.

1.2 What Makes This Design Special

The wings beat in a specific pattern called a **metachronal cascade**. This means the four pairs do not all beat at exactly the same moment — each pair is slightly offset in time from the next. This is the same pattern used by dragonflies, which are among the most aerodynamically capable insects in nature. Dragonflies can hover, fly backwards, turn 180 degrees in three wingbeats, and achieve speeds up to 54 km/h. The MCO-1D is designed to replicate and extend these capabilities.

1.3 The Purpose — Search and Rescue

The MCO-1D is designed for Search and Rescue (SAR) operations. It needs to be able to: fly to a location quickly, hover or fly slowly over a target area while a camera searches, manoeuvre in confined spaces such as between buildings or in wooded terrain, and land or deploy equipment with precision. The combination of flapping-wing efficiency and the ability to fly at very slow speeds — capabilities that conventional drones cannot easily combine — makes it well suited to this role.

1.4 The Three Main Systems

Everything in this project falls into one of three categories:

- **Mechanical:** The wings, the fuselage structure, the sector wheels that drive the wing motion, the ropes and capstans, and the landing gear.
- **Electrical:** The battery, the motor controllers (called ODrives), the motors that drive the wings, the servo motors that control wing angle, the autopilot board (Pixhawk), and the load cells used for testing.
- **Software:** The Wave Library running on the Raspberry Pi, the ground station application on the laptop, and the autopilot firmware on the Pixhawk.

This manual covers all three in the order you need to work through them.

SECTION 2 — HARDWARE INVENTORY — EVERY PART LISTED

Before connecting anything, confirm you have every item in this list. If anything is missing, do not proceed — working with incomplete hardware can damage components.

2.1 Airframe Components

Wings	8 total (4 pairs). Each wing is built from carbon fibre spars and a membrane. Wings 1 and 2 are the largest (front pair), Wing 4 pair is the smallest.
Fuselage	The main body spine. All other components attach to this.
Sector wheels	8 total, one per wing. These are the disc-shaped mechanical components at each wing root that the rope wraps around to drive the flapping motion. Each has 200 degrees of total travel (±100 degrees from centre).
Ropes/Capstans	The mechanical link between the motor shaft and the sector wheel. Must be tensioned correctly.
Landing gear	Three-point wheeled undercarriage. Allows ground roll and taxi.
Wing root joints	The pivot point for each wing. Allows the wing to sweep (fold forward/aft) and change dihedral (tilt up/down).

2.2 Motors

Flap motors	8 × GoolRC 5010 360KV brushless motors. One per wing. These drive the flapping motion through the sector wheel. 360KV means the motor spins at 360 RPM per volt applied.
Motor purpose	The flap motors are the primary thrust and lift producers. They are the equivalent of the engine on a conventional aircraft.

2.3 Motor Controllers (ODrives)

An ODrive is a precision motor controller. It takes commands from the Raspberry Pi and converts them into exactly the right electrical signals to make the motor move to a specific position at a specific speed. Think of it as the interface between the computer (which speaks digital) and the motor (which speaks electrical current).

ODrive units	4 units, each controlling 2 motors (one per channel). Total: 4 ODrives × 2 channels = 8 motor channels.
Communication	CAN-FD bus. This is a high-speed digital communication standard, the same type used in modern cars. All 4 ODrives share a single cable pair running in a daisy-chain.
Power input	4S LiPo battery, approximately 14.8V. Each ODrive has its own power connection from the battery distribution board.

Key setting: vel_limit	Must be set to 150 rad/s before first flight. This is the maximum angular velocity of each motor shaft. Factory default is much lower and must be changed. Procedure in Section 8.

2.4 Servo Motors

Servo motors are small, precise positioning motors. They receive a pulse-width signal and move to a specific angle. Unlike the flap motors, servos do not spin continuously — they move to a commanded position and hold it.

Pitch servos	8 × Helidirect TORQ TCM-1001. One per wing. These rotate the wing chord to change the angle of attack — the angle at which the wing meets the air. Controls how much lift each stroke produces. Operate at 333Hz PWM signal, 7.4V supply.
Sweep servos	8 × DS3225-class servos. One per wing. These fold the wing forward or backward (sweep angle). Used to change the wing's shape for different flight modes. Operate at 50Hz PWM.
PWM signal	Pulse Width Modulation. The servo reads a signal that pulses between 1000 microseconds (full one direction) and 2000 microseconds (full other direction), with 1500 microseconds as neutral.

2.5 Flight Controller (Pixhawk)

The Pixhawk is the autopilot board. It contains an Inertial Measurement Unit (IMU) — a combination of accelerometers and gyroscopes — that measures the aircraft's attitude (which way it is pointing) and motion at all times. It also has a barometric altimeter, GPS, and communicates with the Raspberry Pi.

IMU	Measures roll (banking), pitch (nose up/down), and yaw (turning). Updates at 400Hz.
Barometer	Measures altitude by air pressure. Accurate to approximately ±3 metres.
GPS	Provides position, groundspeed, and heading. Requires clear sky view to acquire satellites.
Communication to Pi	Serial UART at 921600 baud, using MAVLink protocol.
Power	5V regulated, from the power module.

2.6 Raspberry Pi

The Raspberry Pi is the main flight computer. It runs the Wave Library software that controls everything. The Pixhawk provides sensor data; the Pi processes it and sends commands to the ODrives and servos. Think of the Pi as the pilot and the Pixhawk as the instruments.

Model	Raspberry Pi 4 (recommended) or Pi 3B+. Pi Zero 2W can work but needs USB-ethernet adapter.

Storage	MicroSD card, minimum 32GB, Class 10 or better. This holds the operating system and the Wave Library.
Power	5V, 3A minimum. Use the official Pi power supply or a quality USB-C supply.
GPIO pins	The 40-pin header on the Pi connects to the servo signal wires. Specific pin assignments are in Section 3.
Ethernet port	Used for laptop connection during testing. Direct cable to laptop — no router needed.

2.7 Load Cells (for TC-02 and TC-03)

A load cell is a precision force-measuring sensor. During the tethered thrust tests it measures exactly how much upward force the aircraft is producing. This is how we know whether the aircraft can carry its own weight.

Load cell type	HX711-based strain gauge load cell. Rated capacity must exceed the aircraft weight (34N minimum — recommend 50N or 100N rated cell).
HX711 amplifier	The HX711 is a small circuit board that amplifies the tiny signal from the load cell to a level the Pi can read.
Connection to Pi	Two GPIO pins: DT (data) on GPIO 26, SCK (clock) on GPIO 7. Four wires total (DT, SCK, VCC 3.3V, GND).
Mounting	The aircraft hangs or sits on the test rig. The load cell is between the aircraft and the fixed structure. As the wings produce lift, the force reading increases.

2.8 Battery

Type	4S LiPo (Lithium Polymer). 4S means four cells in series.
Voltage	Nominal 14.8V. Fully charged: 16.8V. Do not discharge below 14.0V (3.5V per cell).
Capacity	Recommend 4000-6000mAh for useful flight time. Larger = heavier = less performance.
Connector	XT60. The battery connects to the power distribution board which then feeds the ODrives, Pi, and Pixhawk separately.
WARNING	LiPo batteries are potentially dangerous if punctured, overcharged, or discharged too far. Always use a proper LiPo charger. Never leave charging unattended. Store in a LiPo-safe bag.

2.9 Laptop (for Ground Station)

Any laptop	Mac, Linux, or Windows. Must have Python 3.8 or later installed.

Ethernet port	Needed for direct cable connection to Pi. If laptop has no ethernet port, a USB-to-ethernet adapter works fine.
Software needed	Only Python 3.8+. The ground station application (mco_ground_station.py) uses only standard Python libraries — nothing to install.

SECTION 3 — ELECTRONICS WIRING — WHAT CONNECTS TO WHAT

■ BEFORE CONNECTING ANYTHING: Remove the battery. Work with power disconnected at all times until Section 10 (calibration) specifically instructs you to connect power. Connecting power to wired electronics can cause permanent damage.

3.1 Overview — The Signal Flow

Understanding signal flow makes the wiring logical. Here is the path a command takes from pilot input to physical wing movement:

- Pilot moves lever on transmitter
- Transmitter sends radio signal to receiver
- Receiver sends signal to Pixhawk
- Pixhawk sends sensor data to Raspberry Pi (via serial)
- Raspberry Pi runs Wave Library, computes wing commands
- Pi sends position commands to ODrives (via CAN-FD)
- Pi sends angle commands to servos (via GPIO PWM)
- ODrives drive flap motors
- Servos move pitch and sweep angles
- Wings move — aircraft flies

3.2 CAN-FD Bus — Connecting the ODrives

CAN-FD (Controller Area Network, Flexible Data rate) is a two-wire communication bus. All four ODrives share a single pair of wires (CAN-H and CAN-L) running in a daisy-chain from the Pi to ODrive 1, then to ODrive 2, then ODrive 3, then ODrive 4.

Wires needed	Twisted pair, 24AWG minimum. Use CAT5 ethernet cable — it is already twisted.
Pi CAN interface	MCP2515 CAN hat for Raspberry Pi, or a USB-to-CAN adapter (PCAN-USB or similar).
ODrive CAN pins	Labelled CAN-H and CAN-L on each ODrive. Connect H to H and L to L throughout the chain.
Termination resistors	120 ohm resistors at BOTH ends of the CAN chain. The Pi end and the last ODrive end. Without these the signal reflects and communication fails. Most CAN hats have a built-in termination jumper.
ODrive CAN IDs	Each ODrive must have a unique CAN node ID. Set via ODrive Tool software before installation. IDs 0, 1, 2, 3 for the four units.
CAN speed	Set to 1000000 (1 Mbit/s) on all devices including the Pi CAN interface.

■ CAN-FD is robust and noise-resistant. If communication seems intermittent, check the termination resistors first — this is the most common cause of CAN problems.

3.3 Servo Connections to Raspberry Pi GPIO

The Raspberry Pi communicates with servos via GPIO (General Purpose Input Output) pins. Each servo needs one signal wire connected to a specific GPIO pin on the Pi. All servos also need power (7.4V for pitch servos, 5-6V for sweep servos) and ground — these come from a separate BEC (Battery Eliminator Circuit), not from the Pi.

■ **Never connect servo power (7.4V) to the Pi GPIO pins. GPIO pins are 3.3V signal only. Power goes to the servo rail separately.**

Pitch Servo Pin Assignments (GPIO BCM numbering):

Wing 1 Port (W1p)	GPIO 18
Wing 1 Starboard (W1s)	GPIO 23
Wing 2 Port (W2p)	GPIO 24
Wing 2 Starboard (W2s)	GPIO 25
Wing 3 Port (W3p)	GPIO 12
Wing 3 Starboard (W3s)	GPIO 16
Wing 4 Port (W4p)	GPIO 20
Wing 4 Starboard (W4s)	GPIO 21

Sweep Servo Pin Assignments:

Wing 1 Port sweep	GPIO 4
Wing 1 Starboard sweep	GPIO 17
Wing 2 Port sweep	GPIO 27
Wing 2 Starboard sweep	GPIO 22
Wing 3 Port sweep	GPIO 5
Wing 3 Starboard sweep	GPIO 6
Wing 4 Port sweep	GPIO 13
Wing 4 Starboard sweep	GPIO 19

■ BCM (Broadcom) numbering is used — NOT physical pin numbers. BCM 18 is physical pin 12. Always verify with a Pi GPIO pinout diagram before connecting.

3.4 Load Cell (HX711) to Pi

HX711 DT (Data)	Pi GPIO 26 (physical pin 37)

HX711 SCK (Clock)	Pi GPIO 7 (physical pin 26)
HX711 VCC	Pi 3.3V (physical pin 1 or 17)
HX711 GND	Pi GND (physical pin 9, 25, or 39)
Load cell wires	4-wire load cell: Red=Excitation+, Black=Excitation-, White=Signal+, Green=Signal-. Connect to matching HX711 terminals.

3.5 Pixhawk to Raspberry Pi

Connection type	Serial UART (TELEM2 port on Pixhawk)
Pi side	GPIO 14 (TXD) and GPIO 15 (RXD) — the Pi's hardware serial port, or a USB serial adapter
Pixhawk TELEM2	6-pin JST-GH connector. Pin 2 = TX (connects to Pi RX). Pin 3 = RX (connects to Pi TX). Pin 6 = GND (connects to Pi GND). TX and RX always cross: Pixhawk TX → Pi RX.
Baud rate	921600 baud. Set in both Mission Planner (Pixhawk) and the Wave Library.
Power	Do NOT power the Pi from the Pixhawk or vice versa. Each has its own regulated supply.

3.6 Laptop to Raspberry Pi (Ground Station)

Cable	Standard ethernet cable (Cat5e or Cat6), any length. No crossover cable needed — modern network adapters auto-negotiate.
Pi IP address	192.168.7.1 (set as static)
Laptop IP address	192.168.7.2 (set as static)
Port	5760 (TCP). The ground station server listens on this port.
No router needed	This is a direct point-to-point connection. No WiFi, no router, no internet required.

■ A direct ethernet cable is strongly preferred over WiFi for TC-02 and TC-03. Motor electrical noise can interfere with WiFi and cause dropouts at the worst possible moment.

3.7 Power Distribution

The battery connects to a power distribution board (PDB). From the PDB:

- ODrives: direct 4S LiPo voltage (14.8V). Each ODrive has its own XT30 or XT60 connector.
- Pitch servos: 7.4V regulated BEC. The Helidirect TCM-1001 servos require 7.4V.
- Sweep servos: 5-6V regulated BEC.

- Raspberry Pi: 5V, 3A regulated supply. USB-C connection. Use a quality regulated supply — the Pi is sensitive to voltage fluctuations.
- Pixhawk: 5V from the Pixhawk power module, which also measures battery voltage and current.

■ Connect the battery LAST after all other wiring is complete and verified. Disconnect the battery FIRST before any wiring changes.

SECTION 4 — SOFTWARE — WHAT IT IS AND HOW IT WORKS

4.1 The Three Software Components

There are three pieces of software in this system. They run on different computers and do different things.

Wave Library (mco_wave_library.py)	Runs on the Raspberry Pi. This is the main brain — 24,000+ lines of Python that control every aspect of flight. Reads sensors, computes wing commands, runs the optimizers, manages all safety systems. This is what we have been building throughout this project.
Ground Station (mco_ground_station.py)	Runs on the laptop. A terminal application that connects to the Pi over ethernet and shows you live data from the aircraft. Lets you send commands to the Pi, monitor test progress, and control the optimizer.
Pixhawk Firmware (ArduPilot or PX4)	Runs on the Pixhawk autopilot board. Pre-built firmware — you do not write this code. It provides sensor data (IMU, GPS, barometer) to the Pi via MAVLink protocol.

4.2 What the Wave Library Does

The Wave Library runs a continuous loop — called a "tick" — 50 times per second (every 20 milliseconds). Each tick it:

- Reads all sensor data from the Pixhawk (attitude, airspeed, altitude, GPS position)
- Reads load cell force measurement (during tests)
- Reads pilot control inputs (lever, stick, rudder)
- Runs safety checks (stall speed, never-exceed speed, ground clearance)
- Computes the optimal wing configuration for the current flight condition
- Runs the adaptive optimizers (Loops 1-7) that improve performance over time
- Sends position commands to the ODrives
- Sends angle commands to the pitch and sweep servos
- Sends telemetry to the ground station laptop

This cycle repeats 50 times every second, continuously, for the entire duration of flight.

4.3 The Optimizer System

The Wave Library contains seven adaptive optimizer loops (Loop 1 through Loop 7). These are learning systems that observe how the aircraft is performing and make small adjustments to improve it over time. They do not change anything dramatically — they make tiny, measured changes and observe the result, like a pilot making a series of small trim adjustments. The optimizers improve the aircraft's performance with every flight.

Loop 1 (Stroke level)	Runs every wing stroke. Adjusts the angle of attack (AoA) on the downstroke and upstroke to maximise lift efficiency.
Loop 2 (Cascade phase)	Runs every 2 seconds. Adjusts the timing offset between wing pairs to recover energy from the wake of the front pair (like a coaxial helicopter recovering swirl).
Loop 3 (Surge)	Runs every 5 seconds. Adjusts how the force is distributed between front and rear wing pairs.
Loop 4 (Banked config)	Runs every 12 seconds during banked flight. Adjusts the split-stroke geometry for knife-edge and banked turns.
Loop 5 (Yaw modality)	Runs every 15 seconds. Discovers the best combination of crab, dart, split-stroke, and proverse yaw for efficient turning.
Loop 6 (Lever map)	Runs every 30 seconds. Evaluates whether any discovered wing configurations outperform the current lever zone assignments and proposes remaps.
Loop 7 (Lever feel)	Runs every 45 seconds. Adjusts the sensitivity of the wave lever to minimise how often the pilot needs to make a correction — making the aircraft feel predictable and smooth.

SECTION 5 — SETTING UP THE RASPBERRY PI

5.1 Install the Operating System

The Raspberry Pi needs a Linux operating system. We use Raspberry Pi OS (formerly called Raspbian). This is a free download. Follow these steps exactly.

Step 1 — Download Raspberry Pi Imager:

- Go to: https://www.raspberrypi.com/software/
- Download the Imager for your laptop's operating system (Windows, Mac, or Linux).
- Install and open the Imager application.

Step 2 — Flash the MicroSD card:

- Insert the MicroSD card into your laptop (use an adapter if needed).
- In Raspberry Pi Imager: Choose Device: Raspberry Pi 4 (or your model). Choose OS: Raspberry Pi OS (64-bit). Choose Storage: your MicroSD card.
- Click the gear icon (settings) before writing. Set hostname: mco-1d. Enable SSH. Set username: pi. Set password: choose something memorable. Configure WiFi if you want it (optional).
- Click Write. Wait for it to complete (5-15 minutes).
- Remove the MicroSD and insert it into the Pi.

Step 3 — First boot:

- Connect the Pi to a monitor, keyboard, and mouse (only needed for first setup).
- Connect power. The Pi will boot in 1-2 minutes.
- If you set up SSH, you can also connect from your laptop using: ssh pi@mco-1d.local

5.2 Configure Static IP Address

The Pi needs a fixed (static) IP address so the laptop can always find it at the same address regardless of which network it is on.

Open a terminal on the Pi (or SSH in) and run:

```
sudo nano /etc/dhcpcd.conf
```

Scroll to the bottom of the file and add these lines:

```
interface eth0 static ip_address=192.168.7.1/24 static routers=192.168.7.1 static
domain_name_servers=8.8.8.8
```

Save with Ctrl+X, then Y, then Enter. Reboot:

```
sudo reboot
```

After reboot, verify with:

```
ip addr show eth0
```

You should see 192.168.7.1 in the output.

5.3 Install Required Python Packages

The Wave Library needs a few Python packages. Run these commands in the Pi terminal:

```
sudo apt update sudo apt install python3-pip python3-rpi.gpio -y pip3 install pigpio smbus2 numpy --break-system-packages
```

Start the pigpio daemon (needed for servo PWM):

```
sudo systemctl enable pigpiod sudo systemctl start pigpiod
```

Verify pigpio is running:

```
sudo systemctl status pigpiod
```

You should see "active (running)" in green.

5.4 Install Python CAN Library

The Wave Library communicates with the ODrives over CAN-FD. Install the CAN library:

```
pip3 install python-can odrive --break-system-packages
```

Enable the SPI interface (needed for the MCP2515 CAN hat):

```
sudo raspi-config
```

Navigate to: Interface Options → SPI → Enable. Then reboot.

SECTION 6 — LOADING THE WAVE LIBRARY ONTO THE PI

6.1 Transfer the Files

You need to copy two files from your development computer to the Raspberry Pi:

- mco_wave_library.py — the main flight software
- mco_ground_station.py — the ground station client (this actually runs on the laptop, but keep a copy on the Pi too)

Method 1 — Using SCP (secure copy) over the network:

```
# Run this on your laptop, not the Pi scp mco_wave_library.py pi@192.168.7.1:~/ scp
mco_ground_station.py pi@192.168.7.1:~/
```

Method 2 — Using a USB drive:

- Copy files to a USB drive on your laptop.
- Plug the USB drive into the Pi.
- On the Pi: sudo cp /media/pi/USBDRIVE/mco_wave_library.py ~/

6.2 Verify the Library Loads

SSH into the Pi and run:

```
cd ~ python3 -c "import mco_wave_library as w; print('Library OK')"
```

If you see "Library OK" the file loaded without errors. If you see error messages, check that all required packages from Section 5.3 are installed.

6.3 How to Interact with the Library

The Wave Library is a Python module. You interact with it by writing short Python scripts that import it, or by typing commands in the Python interactive interpreter. During TC-02 and TC-03 you will use pre-written test scripts.

To start an interactive session on the Pi:

```
python3 >>> import mco_wave_library as w >>> w.gs_start() # starts ground station server
>>> w.gnd_clear_status() # shows ground clearance status >>> exit() # to quit
```

The most important functions during testing are:

w.gs_start()	Start the ground station TCP server. Call this first — it lets the laptop connect.
w.aero_cal_status()	Show aerodynamic calibration status. How many observations have been collected.
w.lm6_zone_report()	Show the current lever zone map and any pending remap candidates.

w.violent_manoeuvre_status()	Show which safety systems are currently active.
w.l7_status()	Show the lever feel optimizer — how smooth the lever feels and how it is being tuned.

6.4 The TC-02 Test Script

Create a file on the Pi called run_tc02.py:

```
import mco_wave_library as w # Start ground station so laptop can monitor w.gs_start() # Connect hardware (adjust these for your setup) import pigpio pi = pigpio.pi() # Run the static thrust test # Parameters: pigpio instance, ODrive axes list, # load cell reader, output CSV path results = w.tc02_static_thrust_test( pi_handle=pi, odrive_axes=None, # replace with actual axes loadcell=None, # replace with HX711 reader ) # Feed results into calibration system w.tc02_feed_calibration(results['results']) w.aero_cal_status() pi.stop()
```

Run it with:

```
python3 run_tc02.py
```

SECTION 7 — SETTING UP THE LAPTOP GROUND STATION

7.1 What the Ground Station Shows

The ground station application (mco_ground_station.py) runs on your laptop and shows a live display of everything happening on the Pi. It refreshes automatically every half second. You can also type commands to send to the Pi.

The display is divided into panels:

- **Flight State:** airspeed, height, specific excess power (Ps — positive means climbing, negative means descending), bank angle, flap frequency, lever position. Also shows which systems are currently active.
- **TC-02/TC-03 Data:** each test point displayed the instant the load cell reading completes. Shows lift force in Newtons, CT (thrust coefficient), and whether T/W (thrust-to-weight) exceeds 1.0.
- **Lever Zone Map:** current zone assignments and any modes the optimizer is proposing to add.
- **Command prompt:** type commands here to control the Pi remotely.

7.2 Configure the Laptop Static IP

The laptop needs a static IP on the same subnet as the Pi (192.168.7.x).

On macOS:

- System Settings → Network
- Select your ethernet adapter (or USB ethernet)
- Click Details
- TCP/IP tab → Configure IPv4: Manually
- IP Address: 192.168.7.2
- Subnet Mask: 255.255.255.0
- Router: (leave blank)
- Click OK and Apply

On Linux:

```
sudo ip addr add 192.168.7.2/24 dev eth0 # Replace eth0 with your actual interface name #
Find it with: ip link show
```

On Windows:

- Control Panel → Network and Internet → Network Connections
- Right-click your ethernet adapter → Properties
- Select "Internet Protocol Version 4 (TCP/IPv4)" → Properties
- Select "Use the following IP address"
- IP: 192.168.7.2 Subnet: 255.255.255.0 Gateway: (blank)
- Click OK

7.3 Verify the Connection

With the ethernet cable connected and both IPs set, test from the laptop:

```
ping 192.168.7.1
```

You should see responses like "64 bytes from 192.168.7.1". If you see "Request timeout" or "No route to host", check: cable connected, Pi is on, IP addresses correct.

7.4 Install Python on the Laptop

Check if Python 3.8+ is already installed:

```
python3 --version
```

If you see Python 3.8 or higher, you are ready. If not, or if python3 is not found:

- Mac: Install from https://www.python.org/downloads/ or use Homebrew: brew install python3
- Linux: sudo apt install python3
- Windows: Download from https://www.python.org/downloads/ and run the installer. Check "Add to PATH".

7.5 Run the Ground Station

Copy mco_ground_station.py to your laptop. Then, with the Pi running and gs_start() called on the Pi:

```
python3 mco_ground_station.py 192.168.7.1
```

You will see "Connected." and the display will appear. The command prompt shows "mco>" where you type commands.

Essential commands to know:

status	Full telemetry snapshot from the Pi
aero	Aerodynamic calibration detail — shows what has been measured so far
zones	Lever zone map — shows which wing modes are assigned to which lever positions
freeze	Freeze all optimizer loops — use before changing any settings manually
restore	Unfreeze optimizer loops after changes
set KEY value	Change a setting on the Pi. Example: set WFAOA_GAIN 0.8 set LM6_AUTO_REMAP true
remap ZONE	Confirm a pending lever zone remap. Example: remap SPRINT
abort	Stop a TC-02 or TC-03 test immediately
help	Show all available commands

q	Quit the ground station

SECTION 8 — ODRIVE CONFIGURATION — STEP BY STEP

■ This section requires ODrive Tool software on your laptop and a USB connection to each ODrive one at a time. Do NOT connect the ODrives to the CAN bus until they are individually configured.

8.1 Install ODrive Tool on Laptop

```
pip3 install odrive
```

Verify:

```
odrivetool --version
```

8.2 Connect One ODrive at a Time

Take the first ODrive. Connect only:

- USB cable from ODrive to laptop
- Power supply to ODrive (use bench supply set to 14.8V, limit 2A for configuration — NOT the main battery)
- One motor connected (motor 0 or 1)

Do NOT connect the CAN bus yet.

```
odrivetool
```

You will see "Connected to ODrive..." and a prompt showing the firmware version.

8.3 Configure Each ODrive

In the ODrive Tool prompt, type these commands one by one. Press Enter after each.

Set CAN node ID (use 0 for first ODrive, 1 for second, 2 for third, 3 for fourth):

```
odrv0.can.config.node_id = 0 odrv0.can.config.baud_rate = 1000000
```

Configure motor (do this for both axis0 and axis1):

```
odrv0.axis0.motor.config.pole_pairs = 7
odrv0.axis0.motor.config.resistance_calib_max_voltage = 4
odrv0.axis0.motor.config.requested_current_range = 25
odrv0.axis0.motor.config.current_control_bandwidth = 100
odrv0.axis0.motor.config.motor_type = MOTOR_TYPE_HIGH_CURRENT
```

Set velocity limit — THIS IS CRITICAL:

```
odrv0.axis0.controller.config.vel_limit = 150 odrv0.axis1.controller.config.vel_limit =
150
```

■ The vel_limit MUST be set to 150 rad/s. The factory default is much lower and will prevent the wings from reaching the necessary flapping frequency. Without this change the aircraft cannot generate enough thrust to fly.

Set control mode to position control:

```
odrv0.axis0.controller.config.control_mode = CONTROL_MODE_POSITION_CONTROL
odrv0.axis0.controller.config.input_mode = INPUT_MODE_POS_FILTER
odrv0.axis1.controller.config.control_mode = CONTROL_MODE_POSITION_CONTROL
odrv0.axis1.controller.config.input_mode = INPUT_MODE_POS_FILTER
```

Configure encoder (hall effect sensors):

```
odrv0.axis0.encoder.config.mode = ENCODER_MODE_HALL odrv0.axis0.encoder.config.cpr = 42
odrv0.axis1.encoder.config.mode = ENCODER_MODE_HALL odrv0.axis1.encoder.config.cpr = 42
```

Save and reboot:

```
odrv0.save_configuration() odrv0.reboot()
```

Repeat for all four ODrives, changing the node_id to 1, 2, 3 for each subsequent unit.

8.4 Calibrate Each Motor

With one ODrive powered (bench supply) and one motor connected, run motor calibration:

```
odrv0.axis0.requested_state = AXIS_STATE_MOTOR_CALIBRATION
```

The motor will beep and make a short movement. Wait until it stops. Check for errors:

```
odrv0.axis0.motor.error
```

If the result is 0, calibration succeeded. If non-zero, check motor connections and power supply.

Then calibrate the encoder:

```
odrv0.axis0.requested_state = AXIS_STATE_ENCODER_HALL_POLARITY_CALIBRATION
```

Wait for completion, then:

```
odrv0.axis0.requested_state = AXIS_STATE_ENCODER_OFFSET_CALIBRATION
```

Save the calibration:

```
odrv0.axis0.config.startup_encoder_offset_calibration = True odrv0.save_configuration()
```

■ The calibration only needs to be done once per motor. If you replace a motor, repeat calibration for that channel.

SECTION 9 — PIXHAWK CONFIGURATION — STEP BY STEP

9.1 Install Mission Planner on Laptop

Mission Planner is the configuration software for the Pixhawk. Download from:

```
https://ardupilot.org/planner/docs/mission-planner-installation.html
```

Alternatively, use QGroundControl from:

```
https://docs.qgroundcontrol.com/master/en/getting_started/download_and_install.html
```

Either works. This manual uses Mission Planner examples.

9.2 Flash ArduPilot Firmware

- Connect Pixhawk to laptop via USB.
- Open Mission Planner.
- Go to Setup → Install Firmware.
- Select your vehicle type. For the MCO-1D select "ArduPlane" — closest to our fixed-wing-like control structure.
- Click the appropriate firmware version and wait for installation to complete.
- The Pixhawk will reboot with new firmware.

9.3 Key Parameter Settings

In Mission Planner, go to Config → Full Parameter List. Set these parameters:

SERIAL2_BAUD	921 (= 921600 baud — connection to Pi)
SERIAL2_PROTOCOL	2 (MAVLink 2)
ARSPD_TYPE	1 (analog airspeed sensor) or 0 if using pitot tube with digital output
COMPASS_ENABLE	1 (enable magnetometer)
INS_GYRO_FILTER	20 (gyroscope filter)
AHRS_EKF_TYPE	3 (EKF3 — best for this application)

After setting parameters, click Write Params to save to the Pixhawk.

9.4 Sensor Calibration

In Mission Planner, go to Setup → Mandatory Hardware:

- **Accelerometer Calibration:** Follow the on-screen instructions to hold the Pixhawk in 6 orientations. This calibrates the accelerometer so the IMU knows which way is up.

- **Compass Calibration:** Click Start, rotate the Pixhawk through all orientations slowly. Complete when the progress bar reaches 100%.
- **Radio Calibration:** If using an RC transmitter, move all sticks to their extremes while watching the bar graphs to set the minimum and maximum values.
- **Level Calibration:** Place the Pixhawk level on a flat surface and click Calibrate Level.

SECTION 10 — SERVO CONFIGURATION AND CALIBRATION

10.1 What Calibration Achieves

Servo calibration tells the Wave Library exactly how to convert a commanded angle (in degrees) into a pulse-width signal (in microseconds) for each specific servo. Because servos vary slightly from unit to unit, each of the 16 servos (8 pitch, 8 sweep) must be calibrated individually.

10.2 Pitch Servo Calibration

With the Pi running and servos powered (7.4V BEC), run the calibration function. SSH into the Pi and:

```
python3 >>> import mco_wave_library as w >>> import pigpio >>> pi = pigpio.pi() >>> #
Calibrate wing 1 port pitch servo (index 0) >>> w.pitch_servo_calibrate(pi, wing_index=0)
>>> # Repeat for indices 1 through 7
```

The calibration procedure for each servo:

- The servo moves to the neutral position (1500 microseconds). Observe that the wing is at 0 degrees angle of attack (chord horizontal, aligned with the wing span).
- The function then commands known angles and asks you to confirm or adjust until the physical angle matches the commanded angle.
- Results are stored in the library automatically.

10.3 Direction Verification

After calibration, run the direction verification:

```
>>> w.direction_verify(pi)
```

This commands each wing to move in a specific direction and asks you to confirm it moved the right way. If a wing moves backwards, the servo direction for that wing is reversed in the library automatically.

10.4 Startup Self-Check

Run the startup check to verify all systems:

```
>>> w.mco_startup_check(pi) # This checks all servos, CAN communications, # load cell (if
connected), and sensor readings. # Fix any reported errors before proceeding.
```

SECTION 11 — TC-02 STATIC THRUST TEST — COMPLETE PROCEDURE

11.1 What TC-02 Measures

TC-02 (Test Condition 02) is the static thrust test. "Static" means the aircraft is not moving — it is attached to a test rig and the wings flap while we measure the upward force they produce. This answers the fundamental question: can this aircraft carry its own weight?

The key number is CT — the Thrust Coefficient. This is a dimensionless number that characterises how efficiently the wings produce force. If CT multiplied by the dynamic conditions equals a force greater than the aircraft weight (34N), the aircraft can hover. If not, it needs forward speed to fly.

11.2 Test Rig Setup

Before any electronics are connected:

- Mount the aircraft rigidly to a sturdy test stand. The aircraft must not move during the test — any movement will corrupt the force readings.
- Mount the load cell between the aircraft and the test stand. The cell should be in line with the expected thrust direction (vertically, below the aircraft, so upward thrust compresses or stretches the cell depending on mounting).
- Ensure the wings have clear space to flap — at least 1 metre clearance in all directions.
- Keep all cables (power, CAN, servo signal) clear of the wing paths.

■ During TC-02 the wings flap at up to 3 Hz with 96 degrees of amplitude. Keep hands, tools, and other objects away from the wing paths at all times when the test is running.

11.3 Electronics Connection for TC-02

- Connect load cell to HX711 module.
- Connect HX711 to Pi GPIO (Section 3.4).
- Connect ODrives to CAN bus.
- Connect all 8 pitch servos to GPIO (Section 3.3).
- Connect servo power (7.4V BEC).
- Connect ethernet cable: Pi to laptop.
- Connect main battery LAST.

11.4 Starting the Test

Step 1 — Start ground station on Pi:

```
ssh pi@192.168.7.1 python3 >>> import mco_wave_library as w >>> w.gs_start()
```

Step 2 — Connect laptop ground station:

```
# On laptop python3 mco_ground_station.py 192.168.7.1
```

You should see "Connected" and the live display appears.

Step 3 — Run startup check:

```
# On Pi >>> w.mco_startup_check(pi) # Fix any errors before continuing
```

Step 4 — Run TC-02:

```
>>> results = w.tc02_static_thrust_test( ... pi_handle=pi, ... odrive_axes=axes, # your
ODrive axis list ... loadcell=lc, # your HX711 reader object ... )
```

The test runs automatically. On the laptop you will see each test point appear as it completes — frequency, amplitude, cascade phase, lift force, and CT.

11.5 What to Watch on the Laptop During TC-02

On the ground station display, watch the TC-02 Data panel. For each test point you will see:

Lift N	Force in Newtons. Aircraft weight is 33.8N. If any point shows lift > 33.8N, the aircraft can generate enough thrust to support itself.
CT	Thrust coefficient. Higher is better. Values above 0.20 indicate good thrust performance.
T/W	Thrust-to-weight ratio. Green tick = above 1.0 = more thrust than weight. This is the goal.
Profile	Which wing pitch profile was used — slow (32° AoA), cruise (22°), or climb (28°).

11.6 After TC-02 Completes

The results are automatically fed into the aerodynamic calibration system. Check the updated model:

```
# On laptop ground station mco> aero
```

You will see the updated values for CL_MAX, peak AoA, efficiency AoA, and Vs (stall speed). These values now reflect what the actual aircraft produces, not just what the aerodynamic models predicted.

The TC-02 sweep test also runs automatically after the main test — this tests different wing sweep angles and dihedral combinations to validate the glide polar model.

If TC-02 shows T/W < 1.0 at all conditions:

- The aircraft cannot hover in free air in the conventional sense.
- Forward flight is still possible — the aircraft needs airspeed to generate enough lift.
- The STOL and inclined stroke plane modes may still provide near-hover capability — this is tested in TC-03 with the aircraft pitched at 52 degrees nose-up.
- Do not be discouraged — many successful ornithopters do not hover and are still highly capable aircraft.

SECTION 12 — TC-03 DYNAMIC THRUST TEST — COMPLETE PROCEDURE

12.1 What TC-03 Measures

TC-03 measures how the thrust changes as the aircraft moves forward through the air. In TC-02 the aircraft was stationary. In TC-03 it moves at different speeds (typically 0 to 15 m/s in increments). This produces the CT-J curve — a graph showing how thrust coefficient (CT) varies with advance ratio (J). J is the ratio of forward speed to wing-tip speed. When J is low (slow forward speed, fast wings) efficiency is high. As J increases (faster forward flight) the thrust coefficient typically decreases.

TC-03 is usually done by mounting the aircraft on a vehicle (car or cart) and driving at different speeds while measuring force. An alternative is a wind tunnel if one is available.

12.2 TC-03 Procedure Summary

- Mount aircraft on vehicle with load cell, same as TC-02 but on a moving vehicle.
- Drive at each test speed (2, 4, 6, 8, 10, 12, 15 m/s) and hold steady for at least 10 seconds.
- The load cell records force at each speed.
- Results automatically feed into the stroke plane controller model.

```
>>> results = w.tc03_dynamic_thrust_test( ... pi_handle=pi, ... odrive_axes=axes, ...
loadcell=lc, ... pitot_fn=pitot_reader, # function returning m/s ... )
```

After TC-03, the full aerodynamic model is calibrated to this specific aircraft at its actual weight and with its actual wings. All optimizer loops now have accurate baseline data.

SECTION 13 — FIRST FREE FLIGHT — CHECKLIST AND PROCEDURE

■ Do not attempt free flight until TC-02 and TC-03 are complete and all calibrations have been verified. Do not fly in winds above 5 m/s for first flights. Always fly in a clear open area well away from people.

13.1 Pre-Flight Checklist

Complete every item before each flight. Sign off each:

■ Battery voltage	> 15.0V (fully charged = 16.8V)
■ All servo connections	Verified, not loose
■ CAN bus connections	Verified, terminators in place
■ Wing hinges	Free movement, no binding
■ Rope/capstan tension	Correct — not slack, not overtight
■ Load cell removed	Test rig disconnected for free flight
■ Pixhawk GPS lock	At least 8 satellites, HDOP < 2.0
■ Startup check passed	w.mco_startup_check() — all green
■ Ground station connected	Laptop shows live telemetry
■ Area clear	No people within 30 metres

13.2 First Flight Sequence

Flight 1 should be the minimum possible to verify basic controllability. Approximately 30-60 seconds.

- **Low hover/taxi test:** With the wave lever at approximately 50% (trim), advance slowly and observe whether the aircraft lifts off. If it does not lift off, this confirms forward flight is required and you proceed to a takeoff run.
- **First takeoff:** Advance the wave lever toward 60-70%. The aircraft should begin to lift. Allow it to climb to approximately 5 metres AGL. Do not attempt any manoeuvres.
- **Level flight check:** Find the lever position where the aircraft flies level (Ps near zero on the ground station). This confirms the trim point.
- **Landing:** Reduce lever slowly. As airspeed drops below 6.3 m/s the STOL back-side controller activates (shown on ground station). Use the lever to control descent rate. Land gently with lever at minimum.

13.3 Polar Survey (First Flight)

Once the aircraft is confirmed flyable and stable, start the autonomous polar survey on the second flight at a safe height (minimum 30 metres AGL):

```
# On laptop ground station mco> survey 35
```

The aircraft will automatically run through its test sequence — approximately 12 minutes. Do not intervene unless there is a safety issue. The survey updates all aerodynamic model constants from actual flight data.

SECTION 14 — WHAT THE OPTIMIZER IS DOING AND HOW TO READ IT

14.1 Understanding the Optimizer Display

The ground station shows optimizer status at all times. Here is what each indicator means:

Ps (Specific Excess Power)	The most important single number. Positive Ps means the aircraft has more thrust than needed to maintain level flight — it is gaining energy. Negative means it is losing energy. At trim, Ps is near zero. Unit: metres per second.
L1-L4 frozen	If any loop shows "frozen" it means a safety event has stopped the optimizer from changing anything. This is correct behaviour during violent manoeuvres. The optimizers resume automatically when safe.
Discovery ON/OFF	When ON, the aircraft is actively experimenting with new wing configurations. This is safe — the safety limits always apply even during discovery. The aircraft will not try configurations that could cause a stall or structural damage.
L6 candidates	The number of lever zone remap proposals the system has accumulated. When this is > 0, the system has found a better mode for one or more lever zones. Type "zones" in the ground station to see details and "remap ZONE" to confirm.
corr/min (Loop 7)	Lever correction frequency — how often the pilot makes a direction reversal on the lever. Lower is better. Target is below 2 corrections per minute. As Loop 7 learns, this should decrease over flights.

14.2 What Normal Looks Like

During a normal stable flight you expect to see:

- Ps between -0.1 and +0.3 during level flight
- All loops running (not frozen)
- Discovery OFF unless you have explicitly enabled it
- corr/min below 3
- CL_MAX and Vs values close to the TC-02 measurements

14.3 When to Intervene

Only intervene in the optimizer if you see:

- Ps consistently very negative (below -1.0) during level flight — indicates something is wrong with the calibration or the aircraft is too heavy.
- L6 proposing to retire a mode you use regularly — inspect the proposal before confirming.
- Discovery mode finding highly unusual configurations — inspect before locking them in.

In all other cases, let the optimizer run. It is improving the aircraft with each flight.

SECTION 15 — EMERGENCY PROCEDURES

■ These procedures must be memorised before first flight. In an emergency there is no time to look them up.

15.1 Loss of Control

- Move wave lever to minimum immediately.
- Level the wings with the roll stick.
- Allow the aircraft to descend and land.
- Do not attempt to recover from unusual attitudes below 15 metres AGL.

15.2 Runaway Aircraft (radio link lost)

The RC watchdog will automatically activate if radio signal is lost for more than 1.5 seconds. The aircraft will enter Return to Home mode and fly back to the launch point at 15 metres altitude, then descend and land.

If the RTH appears to be malfunctioning:

- The aircraft will land automatically after RTH_LAND_TIMEOUT seconds (default 30).
- Stand clear of the landing area.

15.3 Fire / Battery Damage

- Do not touch a burning LiPo battery.
- Use a Class D fire extinguisher (dry sand also works).
- Do not use water on a LiPo fire.
- Evacuate the area — LiPo fires produce toxic fumes.

15.4 Abort TC-02 or TC-03 Test

To immediately stop a running test from the laptop:

```
mco> abort
```

Or from the Pi terminal: Ctrl+C in the test script.

To cut power immediately: disconnect the battery.

SECTION 16 — GLOSSARY — EVERY TECHNICAL TERM EXPLAINED

AoA (Angle of Attack)	The angle between the wing chord (a line from leading edge to trailing edge) and the direction of airflow. Higher AoA = more lift, up to a limit (stall).
Advance Ratio (J)	Forward speed divided by wing-tip speed. J=0 means hovering. Higher J = faster forward flight.
Baud rate	Speed of serial communication, measured in bits per second. 921600 baud = 921,600 bits per second.
BEC (Battery Eliminator Circuit)	A voltage regulator that converts the main battery voltage (14.8V) to a lower regulated voltage (5V or 7.4V) for servos and electronics.
Boyd Cruise	The most efficient forward flight mode. Named after John Boyd (OODA loop creator) whose energy maneuver theory underlies the flight software's energy management.
CAN-FD	Controller Area Network Flexible Data rate. A high-speed two-wire serial bus used to communicate with the ODrives. Robust against electrical noise.
Cascade Phase	The timing offset between consecutive wing pairs. Optimal phase allows each pair to recover energy from the wake of the pair ahead.
CF1 (Clap-and-Fling)	A wing motion technique where the wings come very close together at the top or bottom of each stroke, then fling apart. Produces extra lift through vortex enhancement. Used at low speeds for maximum thrust.
CT (Thrust Coefficient)	A dimensionless number that characterises thrust efficiency. Higher CT = more efficient thrust production.
Dihedral	The upward angle of the wings relative to horizontal. Positive dihedral (tips higher than roots) gives stability. Negative dihedral (anhedral) aids maneuverability.
GEF (Ground Effect Flight)	When an aircraft flies within one wingspan of the ground, the downwash is disrupted and lift increases. The MCO uses this zone deliberately for efficient low-altitude operations.
GPIO	General Purpose Input Output. The 40 programmable pins on the Raspberry Pi used to control servos and read sensors.
HX711	A small amplifier chip that reads load cell signals and converts them to digital values the Raspberry Pi can read.
IFR (Instrument Flight Rules)	Rules governing flight in poor visibility using instruments rather than visual references. The MCO is designed to be capable of precise altitude and speed control to IFR standards.

IMU (Inertial Measurement Unit)	Sensors measuring acceleration and rotation. The Pixhawk's IMU tells the flight computer which way the aircraft is pointing and how fast it is rotating.
Lever / Wave Lever	The pilot's primary thrust control. Analogous to the throttle in a conventional aircraft. Controls energy state.
LiPo	Lithium Polymer battery. High energy density. Used in this aircraft for its light weight and high current output. Requires careful handling.
MAVLink	A lightweight communication protocol used between the Pixhawk and the Raspberry Pi to transmit sensor data and commands.
Metachronal Cascade	The timing pattern where each wing pair beats slightly after the previous one. Creates a wave-like motion through the wing pairs. Inspired by dragonfly flight.
ODrive	A high-performance motor controller. Takes position commands from the Pi and drives the flap motors with precise current control.
Pitot Tube	A forward-facing tube that measures airspeed by sensing the difference between total pressure (from the airflow) and static pressure.
Proverse Yaw	When the aircraft turns in the same direction as it banks, unlike the adverse yaw of conventional aircraft. Flapping wings naturally produce proverse yaw, which aids coordinated turns.
Ps (Specific Excess Power)	A measure of whether the aircraft has more power than needed for level flight. Positive Ps = excess power available to climb or accelerate. Negative = deficit.
PWM (Pulse Width Modulation)	The signal format used to control servos. A pulse is sent repeatedly; the width of the pulse (in microseconds) determines the servo position.
Sector Wheel	The disc-shaped mechanical component at each wing root that converts motor rotation into wing flapping. Has 200 degrees of total travel (±100 degrees from centre).
Split Stroke	A configuration where port wings use the bottom half of the flapping arc and starboard wings use the top half. Used for knife-edge flight and banked altitude control.
Stall	The condition where the wing angle of attack exceeds the critical angle and lift suddenly decreases. The MCO stall speed is approximately 4.85 m/s.
STOL (Short Takeoff and Landing)	Flight technique using maximum lift at minimum speed for precision takeoff and landing in short distances.
Stroke Plane	The imaginary flat surface through which the wing tips travel during flapping. Tilting the stroke plane redirects the thrust vector.

T/W (Thrust-to-Weight ratio)	Total thrust divided by aircraft weight. T/W > 1.0 means the aircraft can hover. T/W < 1.0 means it needs forward speed to fly.
Vne (Never Exceed Speed)	The maximum speed the aircraft must never exceed for structural safety reasons. For the MCO-1D: 28 m/s.
Vs (Stall Speed)	The minimum speed at which the aircraft can maintain level flight. Below this speed, conventional lift is insufficient to support the aircraft weight. MCO-1D Vs: approximately 4.85 m/s.
WFAOA (World-Frame Angle of Attack Correction)	A system that adjusts pitch servo commands based on bank angle so the wings always generate upward force regardless of how the aircraft is banked.

End of MCO-1D Commissioning Manual

For questions, corrections, or updates to this document, refer to the programme development log.

SECTION 17 — GUST RESPONSE SYSTEM

17.1 Overview

The Wave Library includes a complete eight-layer gust response system that activates automatically whenever turbulence or atmospheric disturbances are detected. The pilot does not need to do anything — the system runs continuously in the background and the aircraft responds to gusts faster than any human pilot could react.

The system operates on four timescales simultaneously, mirroring how dragonflies and birds respond to wind disturbances:

- **Under 10ms (passive):** Wing compliance mode allows wings to deflect slightly beyond their commanded position when a gust hits, absorbing the impulse mechanically before any electronics can respond. This is the equivalent of a bird wing acting as a suspension system.
- **10ms (IMU fast path):** A dedicated thread runs at 100Hz (independent of the main 50Hz control loop) and detects roll acceleration spikes. When a sideways gust tries to roll the aircraft, the split stroke fraction is adjusted within the next wingbeat — before the full tick cycle has even run.
- **20ms (tick-level):** The main control loop applies GR-1 through GR-7 modes — amplitude boosting on downdrafts, amplitude reduction on updrafts, per-wing sweep and dihedral adjustment, cascade phase widening in sustained turbulence, and quadrant independence for diagonal 3D gusts.
- **Seconds to minutes (Loop 8):** The gust environment characteriser tracks turbulence intensity history, detects recurring patterns (thermals, ridge crossings, valley channelling), and pre-positions wing geometry before the next predicted gust arrives.

17.2 The Eight GR Modes

GR-1 Amplitude Shield	Detects downdraft (accel_z drops below 0.8g). All eight wings increase amplitude by up to 20% to generate more upward force. Decays automatically.
GR-2 Roll Asymmetry Shield	Detects sideways roll gust from roll rate sensor. High-side wings increase amplitude and deploy sweep. Low-side wings reduce amplitude. Generates restoring roll moment within one wingbeat. All DOF used.
GR-3 Longitudinal Group Sync	In strong turbulence (intensity > 0.5): front pairs (W1+W2) synchronise their cascade phase, rear pairs (W3+W4) do the same. Maximum simultaneous force.
GR-4 Active Dihedral Boost	In sustained turbulence: all wings gradually increase positive dihedral angle, improving passive roll stability. Decays smoothly when air calms.
GR-5 Wing Tuck Equivalent	Detects strong updraft (accel_z > 1.3g). Reduces amplitude on all wings to shed lift, preventing the aircraft being thrown upward uncontrollably.

GR-6 Cascade Absorption	In sustained turbulence: widens the cascade phase spacing between pairs. Spreads peak forces across more of the stroke cycle, reducing force oscillations.
GR-7 Quadrant Independence	Detects diagonal 3D gusts (simultaneous roll + pitch accelerations). Front-port and rear-starboard wings increase amplitude; opposite quadrant decreases. Three-dimensional torque generation.
GR-8 Pre-Adaptive Morphing	Pattern recognition: if the same gust sequence has occurred 3+ times (e.g. thermals every 8 seconds), the system pre-positions wing geometry 2.5 seconds before the next predicted event.

17.3 Loop 8 — Gust Environment Characteriser

Loop 8 runs every tick and maintains a 90-second history of gust events. It computes a turbulence intensity score (0 to 1) and watches for patterns in the timing and magnitude of events. As the aircraft flies more hours in a given terrain, it learns that terrain aerological signature and begins anticipating gusts before they arrive.

The ground station shows: turb_intensity, pattern (if detected), and next_event_s (seconds until predicted next gust). These are in the GR panel of the telemetry display.

17.4 IMU Fast Path — Starting It

The IMU fast path thread is started by calling its start function with an IMU reader callback. Add this to your startup script after gs_start():

```
>>> import mco_wave_library as w >>> # Define a function that reads IMU data >>> #
(replace with your actual MAVLink or IMU read) >>> def read_imu(): ... # Read from Pixhawk
via MAVLink ... msg = mavlink_conn.recv_match(type="ATTITUDE") ... if msg: ... return
{"roll_dps": math.degrees(msg.rollspeed), ... "accel_z": accel_z_from_imu()} ... return
{"roll_dps": 0.0, "accel_z": 1.0} >>> w.imu_fast_path_start(read_imu)
```

■ The fast path runs at 100Hz by default. To upgrade to 400Hz connect the Pixhawk IMU data-ready pin to a Pi GPIO and replace time.sleep() with GPIO.wait_for_edge() in the thread.

SECTION 18 — NEW FLIGHT MODES

18.1 Backward Flight Mode

The MCO-1D can fly slowly backward by reversing the cascade phase direction and inverting the wing angle of attack. This is useful for SAR operations where the aircraft needs to back away from an obstacle without turning around in a confined space.

How to enter	While hovering or flying very slowly (below 4 m/s), push full aft stick and hold. The aircraft detects this as a backward flight request and automatically inverts the wing kinematics.
Speed limit	Maximum 4 m/s backward. Above this the S1223 airfoil inverted performance is insufficient to maintain altitude. The mode exits automatically if backward speed exceeds limit.
Performance	The S1223 airfoil produces approximately 31% of its normal lift when inverted. The library compensates with increased frequency and reduced amplitude, but backward flight is power-intensive. Expect battery consumption 2-3x normal cruise.
Bank limit	Backward flight is only available within 25° of wings-level. Attempting backward flight at steep bank angles is blocked.
To exit	Release aft stick or advance lever. The aircraft returns to normal forward flight configuration automatically.

18.2 Inclined Hover (Pending TC-02 Confirmation)

■ Inclined hover is not available until TC-02 body pitch test at 52° is completed and inclined_hover_calibrate() is called with the results. The mode is blocked by software until feasibility is confirmed.

Inclined hover means the aircraft pitches its nose up 52° and flaps its wings in a tilted stroke plane. The upward component of the wing force supports the aircraft weight; the forward component provides slow forward motion. This gives a helicopter-like loiter posture where the camera looks forward and slightly down.

After TC-02, call this once to unlock the mode:

```
>>> w.inclined_hover_calibrate( ... tc02_tw_at_52deg=, ... tc02_freq_hz= ... )
```

Required T/W in the stroke plane: 1.623 (= 1/cos(52°)). If TC-02 shows T/W above this, inclined hover is available. If below, the mode reports the minimum forward speed needed.

18.3 Inverted Recovery (Automatic)

If the aircraft rolls past 120° of bank (going inverted), the inverted recovery system activates automatically without any pilot input. The pilot does not need to do anything. The system:

- Phase 1 (0 to 0.1 seconds): confirms attitude, commands maximum frequency, determines which direction to roll back toward wings-level.
- Phase 2 (0.1 to ~1.5 seconds): maximum amplitude asymmetry between port and starboard wings drives a fast roll back toward wings-level. Typically 4-6 wingbeats.
- Phase 3 (bank below 90°): hands off to the high-G pitch-up primitive to recover altitude lost during the inversion.

■ Inverted recovery requires minimum 15m AGL to complete safely. Do not fly inverted manoeuvres below 30m AGL. The system will ABORT if height is insufficient and the aircraft may not recover before ground contact.

SECTION 19 — SOFTWARE UPDATES AND PERFORMANCE

19.1 Current Library Statistics

Library size	27,090 lines, 432 functions, 1.05 MB source
Import time	~323ms on the Pi (once at startup only)
Tick time (Pi 4)	~2ms per tick (10% of 20ms budget at 50Hz)
RAM usage	~2.3MB peak (0.056% of Pi 4 with 4GB RAM)
Storage	1.8MB total on microSD (0.006% of 32GB card)
Threads	5-6 daemon threads, all IO-bound

19.2 Mode Hints System

A new Mode Hints system provides all seven degrees of freedom (sweep, dihedral, AoA, amplitude, cascade phase, split stroke bias, and frequency) as starting-point recommendations for each flight mode. The optimizer uses these as warm starts, reducing convergence time by approximately 50% compared to starting from the nominal defaults each time.

The pilot does not interact with mode hints directly. They operate automatically. On the ground station display, the mode_hints panel shows which mode is currently active and what hint values are being applied.

19.3 Updated Ground Station Commands

gr_status	Show gust response mode states and turbulence intensity history
l8_status	Show Loop 8 pattern library and current predictions
split_status	Show split stroke fractions per wing and compliance gain
imu_fast_path_status	Show 100Hz IMU thread status and trigger count
inclined_hover_calibrate	Unlock inclined hover after TC-02 body pitch test
aero_cal_status	Show aerodynamic calibration state (unchanged from v1)
lm6_zone_report	Show lever zone map and pending remap candidates (unchanged)

19.4 Updated Pre-Flight Checklist (additions to Section 13)

■ **IMU fast path started**	Call w.imu_fast_path_start(read_imu) in startup script. Verify: imu_fast_path_status shows Running=True.
■ **Gust response armed**	Ground station shows turb_intensity=0.0 in calm conditions. GR modes will activate automatically when needed.

■ **Mode hints active**	Ground station mode_hints panel shows current intent and sweep/phase hints. Confirms system is reading lever correctly.
■ **Inclined hover (if applicable)**	Only after TC-02: confirm inclined_hover_feasible() returns True. If None: TC-02 not yet run. If False: forward speed required.

19.5 TC-02 Update — Body Pitch Test

The TC-02 procedure now includes a mandatory body pitch sweep at 0°, 15°, 30°, 45°, and 52° nose-up. This directly measures inclined hover feasibility. After the test completes:

```
>>> results = w.tc02_body_pitch_test(pi, axes, lc, tilt_rig_fn) >>> # Find the 52° result
>>> tw_52 = results["results"][(52, 3.0)]["TW"] # at 3Hz >>> freq_52 = 3.0 # frequency
used in test >>> w.inclined_hover_calibrate(tw_52, freq_52)
```

■ If the tilt rig function is None, the test pauses and asks you to manually tilt the rig to each angle and press Enter. A simple tilting platform (plywood wedge at known angles) is sufficient for initial characterisation.

SECTION 20 — HIGH-SPEED SPRINT AND PROGRESSIVE WING STOWING

20.1 The Sprint Speed Problem

A SAR vehicle that cannot reach its destination quickly is less useful than one that can. The MCO-1D has been designed from the outset to combine the slow-flight precision needed for on-site operations with the transit speed needed to reach remote locations quickly. This section describes how the sprint system works and what speeds are achievable.

20.2 High-Speed FSF — Dynamic Sweep Per Stroke

The High-Speed Flap-Sweep-Forward (HSF) system is the key to higher sprint speeds. Rather than holding the wings at a fixed sweep angle during sprint, the sweep servo moves within each individual stroke:

- **Downstroke:** Wings sweep forward (toward 25-55° depending on speed) so the leading edge meets the combined airstream at the optimal angle. This generates a direct forward thrust component rather than relying entirely on stroke-plane tilt.
- **Upstroke:** Wings sweep aft (toward 75-85°) to reduce the drag of the recovery stroke, just as a bird folds its wing on the upstroke.

The sweep angle changes within each stroke cycle are invisible to the pilot. The aircraft simply feels faster and more responsive at the sprint lever position. Thrust improves by an estimated 15-35% over fixed-sweep sprint configurations.

■ The previous sprint_params() function used sweep=90° (wings pointing sideways). This has been replaced by the HSF system. The old 90° configuration is no longer used.

20.3 Progressive Wing Stowing

As airspeed increases, the rear wing pairs are progressively folded aft to reduce drag. The stowed pairs are swept to approximately 100° (past perpendicular) where they are aerodynamically unloaded. Only the front pairs continue to generate lift and thrust.

Below 15 m/s	All 4 pairs active. Normal flight configuration. Stall speed: 5.2 m/s.
Above 15 m/s	W4 pair folds. 3 pairs active (W1, W2, W3). Stall speed rises to 6.4 m/s. Drag reduces significantly.
Above 25 m/s	W3 pair also folds. 2 pairs active (W1 and W2). Stall speed rises to 9.7 m/s. Must not slow below 12 m/s in this configuration.
Above 45 m/s (sprint only)	W2 pair folds. W1 pair only. Stall speed approximately 14 m/s. Only in dedicated sprint mode — not for general use.

■ When wing pairs are stowed the stall speed rises significantly. The system automatically restores all pairs if airspeed drops within 1.25x of the elevated stall speed. The pilot will see a STOW SAFETY alert on the ground station if this happens.

20.4 Wing Pair Role Differentiation

When pairs are active during sprint, each pair is assigned a specific aerodynamic role rather than all doing the same job:

W1 — Front pair (longest)	LIFT pair. High angle of attack, forward sweep, maximum span. Keeps the aircraft flying at the elevated stall speed.
W2 — Second pair	THRUST pair. Moderate angle of attack, dynamic HSF sweep optimised for forward force generation. Provides the majority of forward thrust.
W3 — Third pair	Secondary thrust pair. Similar to W2 but lighter loading. Folds at 25 m/s.
W4 — Rear pair (shortest)	Folds at 15 m/s. Contributes primarily at low speed where all lift area is needed.

This mirrors the dragonfly forewing/hindwing role split, where forewings specialise in lift and hindwings in thrust during fast forward flight.

20.5 Sprint Speed Estimates

All 4 pairs, HSF dynamic sweep	35-50 m/s (126-180 km/h)
3 pairs active (W4 stowed), HSF	40-55 m/s (144-198 km/h)
2 pairs active (W3+W4 stowed), HSF	50-70 m/s (180-252 km/h)
1 pair active (W1 only)	70-100+ m/s (record attempt only)
Structural ceiling (Vne)	28 m/s current — may be raised after TC testing

■ All sprint speeds above the current Vne of 28 m/s require structural validation of the rope drive, sector wheels, and wing fairings at those dynamic pressures. The Vne will be raised incrementally as testing confirms structural integrity. TC-03 will measure the CT-J curve and define where thrust actually falls off with increasing airspeed.

SECTION 21 — AUTONOMOUS MODE DISCOVERY AND LEVER EVOLUTION

21.1 What This System Does

The lever map is not fixed. The aircraft continuously experiments with new wing configurations during normal flight and permanently inserts the best discoveries into the lever map, replacing less effective configurations. Over time the lever map evolves from the library design assumptions toward empirically proven optima for this specific aircraft.

The pilot experiences none of this. The lever still moves between 0% and 100%. Every position still responds smoothly and predictably. The aircraft simply improves with every flight, with no intervention required.

21.2 How Discovery Works

- **Search space (32 dimensions):** The discovery system explores a space that includes global parameters (frequency, cascade phase, global sweep, AoA, amplitude, dihedral) AND per-pair parameters for all four wing pairs independently (per-pair AoA offset, sweep offset, amplitude scale, phase offset, split stroke bias) AND HSF sweep profile shape.
- **Dwell and evaluate:** Each configuration is held for 8 seconds while the system measures mean Ps. If the result beats the previous best at that lever position by more than 8%, the configuration is registered as a discovered mode with a generated name (DISC_0001, DISC_0042, etc.).
- **Promote to lever map:** The discovered mode is immediately inserted into the live lever map at its lever position. All future calls to that lever position return the discovered mode instead of the library default.
- **Continuous competition:** Better configurations continue to displace weaker ones. The lever map never stops improving.
- **Smooth transitions:** All transitions to and from discovered modes use the same bell-curve smooth_step system as library modes. The pilot feels no discontinuity.

21.3 Enabling and Monitoring

Autonomous discovery is controlled from the ground station:

```
auto_disc_enable / auto_disc_disable / auto_disc_status / disc_registry / lever_map_live
```

The auto_disc_status output shows:

Configs tried	Total configurations evaluated since enable
Modes promoted	How many have been inserted into the live lever map
Live map size	Number of lever positions with a discovered mode outperforming the library default
Registry size	Total named discovered modes (including superseded ones)

■ Autonomous discovery requires height above DISCOVERY_MIN_H_M (default 15m AGL) and is automatically suspended during any violent manoeuvre, gust event above threshold, or when stall risk is detected. It resumes automatically when safe conditions return.

SECTION 22 — LOOP 9: META-OPTIMIZER

22.1 What Loop 9 Does

Loops 1-8 are tactical optimizers — each tunes its own assigned parameters within a fixed boundary. Loop 9 is the strategic optimizer. It observes the outputs of all other loops and adjusts the parameters that define those boundaries. Where Loops 1-8 optimise the aircraft, Loop 9 optimises the optimizers themselves.

22.2 Three Objectives in Priority Order

- **1. Safety (highest priority):** Stability margins, stall clearance, gust response adequacy. If a safety event occurs, all performance-category parameters are frozen for 5 minutes. Safety parameters are never frozen — they may need to increase in response to the same event.
- **2. Performance:** Speed, climb rate, sprint capability. Loop 9 learns the actual crossover speeds at which wing stowing becomes beneficial, the actual optimal AoA for HSF at each speed, and raises the frequency ceiling as the hardware proves itself. Performance parameters can only be adjusted when no safety freeze is active.
- **3. Efficiency:** L/D, endurance, power consumption. This is the lowest priority but informs how the aircraft behaves in the cruise and loiter zones where most SAR mission time is spent.

22.3 What Loop 9 Can and Cannot Change

CAN change (with evidence)	Progressive stow crossover speeds, HSF optimal AoA, frequency ceiling (only ever upward), split stroke gain, gust response authority levels, pilot feel parameters.
CANNOT change	Stall speed, sector wheel travel limits, maximum amplitude, stall recovery constants, inverted recovery minimum height, ground effect height, any constant containing LIMIT, MAX, MIN, SAFETY, EMERGENCY, or ABORT.
Maximum change per evaluation	8% of current value. Small, evidence-based adjustments only. Each parameter needs 20 observations before any adjustment.
Evaluation interval	Every 2 minutes of flight. Slow enough that changes are not noticeable to the pilot within a flight session.

22.4 Ground Station Commands

l9_status	Print all parameters Loop 9 can tune, their current values, observation counts, and hard limits.
l9_freeze PARAM	Prevent Loop 9 from changing a specific parameter. Example: l9_freeze STOW_THRESHOLD_W3_MS
l9_unfreeze PARAM	Allow Loop 9 to resume tuning a frozen parameter.

l9_reset PARAM	Reset a parameter to its factory default value and clear all observations for that parameter.
l9_disable	Disable Loop 9 entirely for this session.
l9_enable	Re-enable Loop 9 after disable.

22.5 Updated Software Statistics (Revision 3)

Library size	29,126 lines (1.12 MB source)
Optimizer loops	9 loops (Loop 9 = meta-optimizer added)
Discovery dimensions	32 (was 14 in previous version)
Live lever map	Grows autonomously — up to 200 positions
Tick time (Pi 4)	~2.2ms per tick (11% of 20ms budget)
RAM usage	~2.5MB peak (unchanged from v2)

22.6 Updated Pre-Flight Checklist Additions

■ **Loop 9 status**	Run: l9_status on ground station. Verify no unexpected parameters outside expected ranges. Note any recent adjustments in the log.
■ **Autonomous discovery intent**	Decide before flight whether to enable autonomous discovery. Enable with: auto_disc_enable Disable if you want repeatable baseline performance: auto_disc_disable
■ **Live lever map**	Check lever_map_live to see if any discovered modes are active. If you see unexpected DISC_NNNN entries at positions you will use, review them with disc_registry.
■ **Progressive stow awareness**	If flying above 15 m/s: the aircraft will automatically stow W4. Brief the pilot that stall speed is elevated above 15 m/s and the aircraft must not slow below 8 m/s in that configuration.